Practical Assignment Automated Reasoning 2IMF25

dr.ir. Thomas Neele
Metaforum, room 6.105
email: t.s.neele@tue.nl

September/October 2026

Organizational Information

The practical assignment has to be executed by groups of three persons. Registration in groups must be done via Canvas, at the latest on **9 September 2026**. After that date, incomplete groups will be merged and students who are not in a group will be considered as no longer taking part in the course.

The results of the assignment has to be described in a report that should be submitted as PDF next to the source code of your implementation in a ZIP-file via Ans. For the report the deadline is **16 October 2026**. Additional information about the course can be found at the course page in Canvas.

Report Requirements

The report must be written in LaTeX and have less than 20 pages with a standard page and font size, e.g., Springers `llncs` style or the `article` class. Source code of your implementations should be avoided in the main part of the report, presenting only small relevant excerpts if necessary for illustration purposes. For all variables and formulas documentation is required, explaining the approach and the overall structure. A generic approach is preferred, since this may result in clearer descriptions, increasing the confidence in the correctness of the results. Formulas of more than half a page should not be contained in the report, instead the structure of the formula should be given using abstract notation. From the output of the programs only relevant parts should be contained in the report, and observations on computation time should be reported. The answers on the problems should be motivated, where relevant including tables or pictures is required. In summary, each answer should at least contain:

- Description of the parameters and variables and their types.
- Formulas that capture the problem.
- Description of how the tool was used to obtain the desired answer.
- Visualisation of the results (table, drawing or other).
- Discussion of runtime.

Every report should contain name, student number and email address of each of the authors as well as the number of the group. Each author is considered to be responsible for the full text and all results.

Guidelines for grading

The assignment consists of seven tasks, the weight of each task is given in the description. The grading will mainly take into account the following aspects:

- Clarity and conciseness of the formulas and accompanying descriptions.
- Correctness of the formulas and of the approach and the argumentation behind it.
- Correctness and completeness of the reported results.

If you do not manage to (completely) solve an exercise, clearly explain the limitations of your solution or the obstacles you encountered. An example of the desired style is provided on Canvas.

The calculation of the grade from the number of points scored is not fixed in advance, so solving half the tasks perfectly does not necessarily mean you obtain a 5.5.

Tools and Libraries

Only the following tools or libraries should be used to solve the problems:

- `Z3` (`https://github.com/Z3Prover/z3`) or `Yices` (`http://yices.csl.sri.com/`)

  Both `Z3` and `Yices` are programs for satisfiability modulo theories (SMT). They accept standard SMT format, in particular boolean SAT format. `Z3` provides APIs for several popular programming languages, `Yices` only provides a C API.

- `OxiDD` (https://oxidd.net)

  `OxiDD` is a modern BDD framework written in Rust, with available APIs for Python and C++. Please use the BDDManager (BDDs without complement edges) to solve the assignments.

Each of the problems should be solved using one of these tools and assessed with current versions linked above.

Use of AI

Use of (generative) AI is allowed for obtaining general knowledge, but not for directly solving the given problems or translating code to formulas. Your report is required to include a statement on whether you used AI, and, if so, how. The general guidelines from the TU/e can be found here.

1) Magic Factory

weight 8 %

Six trucks (with rated speed 80km/h) have to deliver obscure building blocks to a magic factory. The distance to the factory is 120 km. Every truck has a max load of 8000 kg and can fit at most ten building blocks. In total, the following has to be delivered:

- Six nuzzles, each of weight 800 kg.
- Twelve prittles, each of weight 405 kg.
- Fifteen skipples, each of weight 500 kg.
- Eight crottles, each of weight 2500 kg.
- A number of dupples, each of weight 600 kg.

If more than two types of goods are present in a truck, it has to drive a a reduced speed of 60 km/h to lower the risk of magic interactions between the goods. Skipples are round and elastic, only two out of the six trucks have facilities for fastening them. Prittles are somewhat valuable: those pallets need to be distributed over *at least* five trucks.

(a) Investigate what is the maximum number of dupples that can be delivered in a single run, and show how for that number all goods may be divided over the six trucks.

(b) Do the same, with the extra information that crottles get too cold if there are less than two dupples in the same truck.

2) PCB layout

weight 14 %

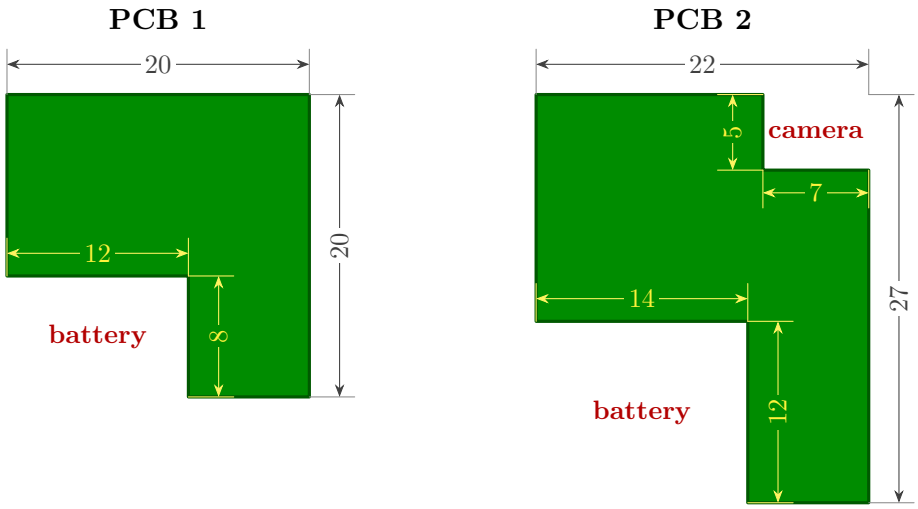
Modern foldable phones contain multiple *printed circuit boards* (PCBs), usually one on each side of the hinge. A phone manufacturer needs to design the internal layout for their next flagship model. And so, they have to fit some chips on these PCBs. What makes it challenging is that the PCBs are odd-shaped in order to fit tightly around the battery and camera. The design should meet the following constraints:

- There are 14 chips that need to fit on the PCBs, each chip is placed exactly once. The dimensions of the PCBs and the size of each chip is given in Figure 1.
- Three of the chips can become warm under load (see again Figure 1). To prevent overheating, their centres should not be closer than 20 length units in either the x or y direction.
- Chips may be rotated 90°, but may not overlap.

(a) Give a layout of the PCBs following the above constraints.

(b) To boost the spec sheet, the camera department would like to integrate more cameras. This will increase the width of the camera cutout by 4 length units for each additional camera. What happens if one camera is added?

(c) And what if three cameras are added?



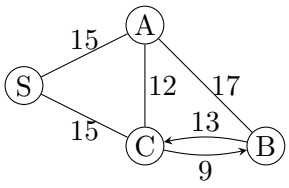
	chips													
width	5	5	4	4	5	3	7	6	6	4	6	5	6	5
height	6	6	6	10	7	7	7	10	12	10	9	11	10	10
hot?	✓	✓				✓								

Figure 1: Layout of both PCBs and and overview of the chips to be fitted.

3) Food Supply

weight 14 %

Three non-self-supporting villages A, B and C consume one food package each per time unit. The required food packages are delivered by a truck, having a capacity of 130 food packages. The truck has to pick up its food packages at location S containing an unbounded supply. The locations of this supply location and the villages and the roads are shown in the picture. Here every number indicates the number of time units the truck needs to travel from one village to another, including loading and delivering. There is often a lot of traffic going from B to C, hence it takes a bit longer than the journey from C to B. The villages only have a limited capacity to store food packages: for A and C this capacity is 90, for B it is 120. Initially, the truck is in S and is fully loaded, and in each of the three villages there are 60 food packages.



(a) Show that it is impossible to deliver food packages in such a way that each of the villages consumes one food package per time unit forever. For how long can the villages be kept alive at best?

(b) Show that this is possible if the capacity of the truck is increased to 150 food packages. What is the shortest possible driving schedule for the truck to sustain the villages indefinitely? Hint: a finite graph contains an infinite path starting in a node v if and only if there is a path from v to a node w for which there is a non-empty path from w to itself.

4) Program Verification

weight 14 %

Consider the following program written in C. Here, `__nondet__()` is a function that non-deterministically returns an arbitrary Boolean value.

```
int main() {
    int a = 1;
    int b = 1;

    while (a < 180) {
        if (__nondet__()) {
            b = b + 3;
            a = a + 2*b;
        } else {
            b = b + a;
            a = a + 5;
        }
    }

    if (b == 190 + k) {
        assert(false);
    }
}
```

Show for each value of $k \in \{0, \ldots, 10\}$ whether the program is safe or can crash (*i.e.* reach the `assert` statement). If a crash is possible, provide one complete run leading to that crash.

5) Configurable Systems

weight 20 %

Almost all software systems are configurable, allowing users to select or deselect specific *features* in their software. Such features are usually modeled with Boolean variables $F = \{x_1, \ldots, x_n\}$ such that a system configuration is an assignment $c\colon F \to \{0, 1\}$ where for each feature $x \in F$, $c(x) = 0$ means that x is deselected in c and $c(x) = 1$ means that x is selected in c. Not all configurations have to lead to an actual software product, e.g., one might disallow for an executable that is both ARM and x86 at the same time. The set of *valid configurations* V is usually provided by a propositional logic formula φ in conjunctive normal form (CNF) where a configuration c is valid (*i.e.* $c \in V$) iff $[\![\varphi]\!](c) = 1$. To compare feature sets, let $F{\restriction_c}$ denote the set of features selected by c, namely $F{\restriction_c} = \{x \in F \mid c(x) = 1\}$.

For some safety-critical software, industrial certification standards are really high. On the other hand, configurable software is complex and activating more features can cause more bugs to appear. To minimise the certification effort, it can be useful to use configurations that select as few features as possible. In this task you have to construct a BDD that represents the set of valid configurations V and then implement the following two techniques for finding small configurations.

(a) A *minimum valid configuration* is a configuration such that there is no other configuration that selects fewer features. More formally, $c_{min} \in V$ is minimum if and only if for all $c' \in V$, it holds that $|F{\restriction_{c'}}| \geq |F{\restriction_{c_{min}}}|$. Note that minimum valid configurations are not necessarily unique.

(b) Some features can be essential and this should be taken into account when creating a minimal configuration. So, generalising the above definition, a valid configuration is *minimum for* $S \subseteq F$ iff it is minimum among all configurations that select all features from S. Formally, $c_{min} \in V$ is minimum for S if and only if $S{\restriction}_{c_{min}} = S$ and for all c' such that $S{\restriction}_{c'} = S$, it holds that $|F{\restriction}_{c'}| \geq |F{\restriction}_{c_{min}}|$.

(c) While finding a single valid configuration can find a good setup, sometimes it is useful to reason about *all* minimum valid configurations. One approach is to capture this again in a BDD.

Describe your implementations, how you find the required minimum valid configurations and how you create the BDDs.

Test your implementations on the five configurable systems given as DIMACS files in the `conf-dimacs.zip` from the course's Canvas folder. See appendix for further descriptions on the input format. For each of the five systems, report on

(i) the total number of BDD nodes used to represent $[\![\varphi]\!]$,

(ii) the total number $|V|$ of valid configurations,

(iii) the size $|F{\restriction}_{c_{min}}|$ of a minimum valid configuration c_{min}.

(iv) the size $|F{\restriction}_{c_{min}}|$ of a minimum valid configuration with respect to the set of essential features $S = \{x_{218}, x_{311}\}$.

(v) The number of BDD nodes used to represent the set of all minimum valid configurations.

(vi) The number of minimum valid configurations.

6) Graph exploration

weight 15 %

One prominent application of *binary decision diagrams (BDDs)* is in representing and reasoning about graphs. In this task, we consider directed graphs with labels on the nodes. More formally, a node-labelled graph is a tuple $\mathcal{G} = (S, T, L)$, where S is a finite set of states, $T \subseteq S \times S$ is a transition relation and $L : S \to \{0, 1\}$ is a labelling function. Note that we restrict ourselves to a binary labelling function here. To denote that $(s, s') \in T$, we also write $s \to s'$.

The Canvas page of the course provides the file `graphs.zip`, containing CSV files representing a few such graphs. One is extracted from Twitter[1] and one from Pokec[2] (a Slovakian social network) and there are two reduced variants of the latter. For each graph, the `_nodes.csv` file lists the nodes and their labels and the `_edges.csv` file lists one transition $s \to s'$ on each line (and possibly other, irrelevant data).

In this task, you have to use BDDs to represent and explore these graphs. Specifically, for each graph, you should

(a) Represent the graph by one or more BDDs choosing a suitable encoding.

(b) Determine how many nodes are reachable when starting from the node with id 1.

[1] `https://snap.stanford.edu/data/congress-twitter.html`
[2] `https://snap.stanford.edu/data/soc-Pokec.html`

(c) Do the same, but now only count nodes that are reachable via alternatingly 0 and 1-labelled nodes. That is count the number of nodes reachable via a path $s_0 \to s_1 \to s_2 \to s_3 \to \ldots$, where s_0 is the node with id 1 (other nodes may have arbitrary identifiers) and, for all i, $L(s_i) = 0$ iff i is even.

Justify your approach. Report on the number of BDD nodes for the BDDs used to solve the task and clearly state what these BDDs represent. As a hint, the graph should be represented by the transition relation and the labelling function. Note that BDD nodes appearing in two component representations, e.g., labelling function and transition relation, should only be counted once. It is recommended to work on the smaller graphs first, since computation for the larger graphs will take a significant amount of time and memory.

7) Challenge

weight 15 %

Give a precise description of a non-trivial problem of your own choice and solve it with the methods taught in the course. Justify the choice of your methods.

Here 'trivial problems' include both minor modifications of earlier problems from the assignment or the lecture, single solutions of simply specified puzzles like sudokus, or problems that can be easily solved by hand. In case of doubt please contact the lecturer.

For the chosen challenge, you have to clearly specify the problem, provide a formal encoding, state the encoding in the implementation, describe the input and output format, and provide the solution along with a justification of its correctness.

Appendix

In this appendix, some information on the input formats for the second assignment is provided. The official format descriptions are provided in the course's Canvas files.

DIMACS

The DIMACS format is the standard format for CNFs, widely used, e.g., in SAT competitions. Essentially, lines in file the stand for clauses, except a heading line starting with `p cnf n m`, which is followed by the number of variables `n` and number of clauses `m`. For instance, in our setting of configurable systems, features correspond to boolean variables and a heading line `p cnf 5 11` describes a system with $n = 5$ features and 11 constraints given as clauses. Clauses are provided through a list of literals that ends with a `0`. For instance, a line `2 -3 5 0` specifies the clause $x_2 \vee \neg x_3 \vee x_5$. For some applications, the variable order $\pi\colon \{x_1, \ldots, x_n\} \to \{1, \ldots, n\}$ is also given in DIMACS files, indicated by a line starting with `c vo ...`. For instance, `c vo 2 3 1` stands for the variable order $\pi\colon \{x_1, x_2, x_3\} \to \{1, 2, 3\}$ where $\pi(x_1) = 3$, $\pi(x_2) = 1$, and $\pi(x_3) = 2$.